MowisAI

Technical and Strategic Overview

Momin Aldahdooh, Co-Founder & CTO Wasay Muhammad, Co-Founder

Tampere, Finland (2026)

Abstract

MowisAI is an AI agent execution engine that enables operators to run many AI agents in parallel with OS-level isolation between each agent. The system is implemented as a single Rust binary and provides an `agentd` daemon that exposes a Unix-domain-socket API plus a multi-layer orchestration layer that coordinates agent tool use via a structured workflow.

Each worker agent operates inside its own isolated Linux filesystem (overlayfs + chroot). Tool execution is performed through the local `agentd` runtime, so code editing, shell execution, Git operations, and file reads/writes occur within the operator's infrastructure. LLM planning and synthesis are executed via a configurable provider (Vertex AI Gemini integration is included in this version); the sandbox execution layer remains local.

This document describes the problem MowisAI solves, its current architecture, the orchestration approach, and the engineering achievements delivered to date.

---

1. Introduction

In modern software development, multiple AI agents are often used concurrently for planning, architecture, implementation, testing, and iterative refinement. The core challenge is that "agentic" workflows become unreliable when agents share a filesystem environment or when their tool execution is not isolated. As concurrency grows, conflicts between agents become predictable failure modes rather than rare edge cases.

MowisAI was designed specifically to address:

- OS-level filesystem isolation so parallel agents cannot corrupt each other.
- Local tool execution through a stable runtime (`agentd`) so tool results remain on-device.
- A scalable orchestration layer that coordinates agents with clear responsibilities.

---

2. Problem Statement

2.1 Agent Conflict in Parallel Execution

When multiple autonomous agents edit or run code in an unpartitioned environment, common failures include:

- Concurrent writes to the same paths causing one agent to overwrite another's work.
- Deleting or mutating shared resources (dependencies, config files, build artifacts).
- Observing inconsistent intermediate states leading to incorrect reasoning.
- A faulty agent corrupting shared runtime state and cascading failures.

2.2 Orchestration Without an Execution Layer

Most orchestration frameworks focus on defining how agents coordinate, but they do not provide an OS-level execution layer. The operator still needs a secure runtime to:

- Create isolated sandboxes.
- Route tool invocations.
- Ensure sandbox and filesystem constraints are enforced.

MowisAI integrates both orchestration and OS-level execution into a single deployable binary.

---

3. Technical Solution

3.1 Architecture Overview (Current)

MowisAI is organized into three layers:

1. **Sandbox layer**: OS-level isolation via overlayfs and chroot.
2. **`agentd` daemon**: local Rust runtime that manages sandboxes/containers and exposes a Unix socket API for tool execution.
3. **Orchestrator layer (multi-layer coordination)**: decomposes work, provisions sandboxes, runs workers, merges results, and synthesizes final delivery.

All orchestration is mediated through `agentd`. Workers and managers invoke tools using structured socket requests.

3.2 Sandbox Layer: overlayfs and chroot

Each agent receives a dedicated sandbox at creation time:

- overlayfs provides copy-on-write isolation on top of a read-only base.
- chroot restricts process filesystem visibility to the sandbox root.

This prevents cross-agent filesystem interference: each agent's filesystem changes remain confined to its own sandbox layers.

3.3 `agentd` Daemon

`agentd` is a Rust daemon that:

- Creates sandboxes and containers.
- Enforces tool execution inside the correct isolated environment.
- Exposes a Unix-domain socket API (`/tmp/agentd.sock` by default).
- Provides a tool registry (75 tools) spanning filesystem operations, shell and script execution, Git, HTTP, memory/secrets, and other utilities.

Tools are invoked by orchestrator/agents via JSON requests over the Unix socket. Responses include structured results (stdout/stderr, success flags, file-operation metadata, and diffs for Git workflows).

3.4 Orchestrator Layer: 5-Layer Architecture (Not LangGraph-style)

The orchestration approach in this version is a scalable layered architecture with explicit responsibilities. It is implemented as a sequence of cooperating "layers," not as a node-based LangGraph mental model.

The layers are:

1. **Context Gatherer** (Layer 1)
   - Uses Gemini to inspect and structure the project state.
   - Produces a structured `ProjectContext` JSON document.
2. **Architect** (Layer 2)
   - Uses Gemini to transform `ProjectContext` into an `ImplementationBlueprint`.
   - Blueprint includes sandbox teams, tool subsets, agent counts, and execution/merge strategy.
3. **Sandbox Owners** (Layer 3)
   - Creates one `agentd` sandbox per blueprint team (sandbox "ownership").
   - Decomposes the sandbox deliverable into per-agent tasks and dependency ordering.
   - Requests stable `agent_id` formatting when possible to enable warm reuse.
4. **Sandbox Managers** (Layer 4)
   - Executes worker tasks in dependency groups.
   - Creates a dedicated merge container (git repo in `/workspace`).
   - Applies worker patch diffs sequentially to merge history.
   - If patch application fails, uses Gemini to repair/produce conflict patches and retries.
5. **Workers** (Layer 5)
   - Runs a tool-calling loop using Gemini in an isolated container.
   - Executes tools via `agentd` socket.

- Collects `git diff` from the worker environment.
- Reports `AgentResult` including changed files and patch content.

3.5 Patch-based Merge + Conflict Repair

Each worker produces a patch (unified diff) derived from the container's Git state. The sandbox manager applies patches into a merge repo:

- `git apply` for fast integration when possible
- if conflicts occur:
  - manager generates a conflict context snapshot
  - Gemini produces repaired patch text
  - manager resets/re-applies and commits when successful

This approach makes the overall system resilient to partial failures and merge conflicts.

---

4. Interactive Mode and Session Persistence (Major Update)

To enable an interactive development workflow (similar to a developer CLI), this version adds:

- `orchestrate-interactive`: keeps the orchestrator process alive for follow-ups.
- **Session persistence**: chat transcript, structured context, and sandbox/container warm-state are saved to disk via a session file.
- **Live coordinator briefing**:
  - On each follow-up turn, a dedicated coordinator step uses Gemini to summarize changes and produce an updated briefing.
  - That briefing is injected into each worker task context so workers remain aligned with the evolving project.
- **Warm reuse**:
  - Merge container state persists to keep Git history available across turns.
  - Worker containers can be reused when the planner produces stable `agent_id`s for the same sandbox team.

Debugging and observability are also first-class:

- Normal CLI mode prints tool invocations and file operations with minimal noise.
- `--debug` enables verbose trace output (HTTP/socket timing, request/response details, model output parts).

---

5. Development Status (As-of March 2026)

As of March 2026, the core execution layer (`agentd`) is production-ready:

- Unix socket server and tool execution paths verified.
- overlayfs + chroot sandboxing enforced at runtime.

The orchestrator layer has been implemented with the 5-layer architecture and supports:

- Vertex AI Gemini integration for planning and synthesis.
- tool-calling worker loops executed through `agentd`.
- patch-based merge workflow with LLM-assisted conflict repair.
- an interactive session mode with session persistence and live coordinator briefing.

Streaming model output, further reduction of token usage, and additional runtime hardening are ongoing.

---

6. Competitive Differentiation

MowisAI differentiates by combining:

- OS-level isolated execution (overlayfs + chroot) with
- multi-agent orchestration in a single deployable Rust binary.

Unlike orchestration-only frameworks, MowisAI includes a secure execution layer. Unlike cloud sandbox providers, the runtime remains on the operator's infrastructure; only LLM calls are made to the configured provider.

---

7. Conclusion

MowisAI addresses a concrete and unsolved gap in AI agent infrastructure: reliable parallel multi-agent execution with OS-level isolation and a scalable orchestration framework delivered as a single Rust binary.

The system is engineered for operators who require strong sandboxing and control over execution environments, with rapid iteration supported by an interactive orchestration CLI and persistent session state.